

The coefficient recursion for the exponential jets

Claude, with Daniel Murfet

2026-09-07

Abstract

The exponential jets $E_n(s) = \sum_{m < R} \frac{(\beta s)^m}{m!} [X^n](g_R^m)$ of unit 94 satisfy

$$nE_n(s) = \beta s \sum_{i=1}^n i x_i E_{n-i}(s), \quad 1 \leq n < R,$$

the computational interface recommended in Astra #3/#4 (the paper's tree recursion in disguise). The proof is finite polynomial algebra: with $T_R = \sum_{m < R} \frac{(\beta s)^m}{m!} g_R^m$, $T'_R - \beta s g'_R T_R = -\frac{(\beta s)^R}{(R-1)!} g'_R g_R^{R-1}$ is divisible by X^{R-1} because g_R is divisible by X , so the coefficients below $R-1$ of T'_R and $\beta s g'_R T_R$ agree. No power-series exponential is needed. Zero `sorry/axiom`.

1 The things formalised

```

noncomputable def expPoly ( : ) (x : → ) (R : ) (s : ) : [X] :=
  m range R, C (( * s) ^ m / (m.factorial : )) * (gPoly x R) ^ m
theorem coeff_expPoly ( : ) (x : → ) (R n : ) (s : ) :
  (expPoly x R s).coeff n = expJet x R n s
theorem X_dvd_gPoly (x : → ) (R : ) : (X : [X]) ∣ gPoly x R
theorem derivative_expPoly ( : ) (x : → ) (R : ) (s : ) :
  derivative (expPoly x (R + 1) s)
    = C (( * s) * derivative (gPoly x (R + 1)))
      * (expPoly x (R + 1) s)
      - C (( * s) ^ R / (R.factorial : )) * (gPoly x (R + 1)) ^ R
theorem coeff_derivative_gPoly (x : → ) (R i : ) (hi : i + 1 < R) :
  (derivative (gPoly x R)).coeff i = ((i : ) + 1) * x (i + 1)
theorem expJet_recursion ( : ) (x : → ) (R n : ) (hn : 0 < n) (hnR : n < R) (s : ) :
  (n : ) * expJet x R n s
    = * s * i range n, ((i : ) + 1) * x (i + 1) * expJet x R (n - 1 - i) s

```

2 Mathematics

Differentiating T_R termwise, $(g^m)' = m g^{m-1} g'$ and $\frac{(\beta s)^m}{m!} m = \beta s \frac{(\beta s)^{m-1}}{(m-1)!}$ reindex the sum to $\beta s g' T_{R-1}$, i.e. $\beta s g' (T_R - \frac{(\beta s)^{R-1}}{(R-1)!} g^{R-1})$. Comparing the coefficient of X^{n-1} : on the left it is $n E_n$ (`coeff_derivative`); on the right, $\beta s \sum_{i+k=n-1} [X^i] g' [X^k] T_R = \beta s \sum_{i < n} (i+1) x_{i+1} E_{n-1-i}$ by the Cauchy product and $[X^i] g' = (i+1) x_{i+1}$, while the correction $g' g^{R-1}$ is a multiple of X^{R-1} (`pow_dvd_pow_of_dvd` from $X \mid g$) and contributes nothing below $R-1$.

3 Paper-facing meaning

Together with the derivative identification of unit 102 this gives the one-dimensional jet coefficients three equivalent descriptions: the closed truncated exponential formula, the normalised derivatives, and the recursion $nE_n = \beta s \sum i x_i E_{n-i}$ that generates them from the Taylor data of ξ alone. The recursion is the natural bridge to the partition formula and to the paper's tree indexing (Astra #4(f)), which remain the only unproved steps of the coefficient identification.

4 Lean notes

- `Polynomial.derivative` lives in `Mathlib.Algebra.Polynomial.Derivative`, not in the project's import closure; import it explicitly.
- Identities mixing `C a * p` with products of polynomials defeat `ring` (`C` of a product is not syntactically a product). Convert to scalar actions with `simp only [C_mul', smul_smul, smul_mul_assoc, mul_smul_comm]`, then `congr 1` separates the scalar identity (`field_simp; ring`) from the polynomial one.
- `coeff_smul + coeff_X_pow_mul'` kill the coefficients of an X^{R-1} -multiple below $R - 1$ in one `rw`.
- `simp` normalises `n' + 1 - 1` to `n'` in the goal but not in a hypothesis introduced later; normalise both (`simp only [Nat.add_sub_cancel] at hkey`) before `linear_combination`.